

Operators and Strings

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Arithmetic Operators

```
5 + 3;    // 8    addition
5 - 3;    // 2    subtraction
5 * 3;    // 15   multiplication
5 / 3;    // 1.666... division - always a float, never integer
5 % 3;    // 2    remainder ("modulo")
5 ** 3;   // 125  exponentiation
```

- `/` never truncates: `10 / 4` is `2.5`, not `2`
- `%` is the one you reach for constantly: even/odd, wrapping, leftovers

Remainder Is More Useful Than It Looks

```
n % 2 === 0;           // is n even?  
1000 % 60;             // 40 - leftover seconds  
(i + 1) % items.length; // wrap to the start of an array
```

- Splitting a total into units is `%` for the leftover, `/` for the whole

```
const total = 1000;  
const seconds = total % 60;           // 40  
const minutes = (total - seconds) / 60; // 16
```

Assignment and Increment

```
let n = 10;  
  
n += 5; // 15    same as n = n + 5  
n -= 3; // 12  
n *= 2; // 24  
n /= 4; // 6  
n %= 4; // 2  
  
n++;    // 3    add one  
n--;    // 2    subtract one
```

- `n++` and `n += 1` do the same thing in a statement of their own

Comparison: `===` versus `==`

```
5 === 5;    // true
5 === "5";  // false - different types, no conversion

5 == "5";   // true  - the string was converted first
0 == "";    // true  - both converted to 0
false == 0; // true
```

- `===` compares value *and* type. Use it everywhere.
- `==` converts first, which produces results nobody predicts
- The same applies to `!==` versus `!=`

Converting Types Explicitly

```
Number("42");    // 42
Number("hello");  // NaN (Not-a-Number)
Number("");       // 0    - a classic surprise

String(42);       // "42"
Boolean(0);       // false
Boolean("");      // false
Boolean("hello"); // true
```

- Converting yourself makes code clearer and easier to debug
- Never rely on `==` to do the conversion for you

Building Strings with

```
const first = "Ada";  
const last = "Lovelace";  
  
first + " " + last; // "Ada Lovelace"
```

-  means **add** for numbers and **join** for strings
- If either side is a string, the other is converted to one

```
"Total: " + 30; // "Total: 30"  
"5" + 3;        // "53"    - not 8!  
"5" - 3;        // 2       - only + is overloaded
```

Template Literals

- The modern way to assemble text with embedded values

```
const name = "Alice";
const count = 3;

// Old way:
"Hi " + name + ", you have " + count + " orders.";

// Modern way - backticks, not quotes:
`Hi ${name}, you have ${count} orders.`;
// "Hi Alice, you have 3 orders."
```

- Backticks also span multiple lines without `\n`

`${}` Holds Any Expression

```
const qty = 3;
const price = 4;

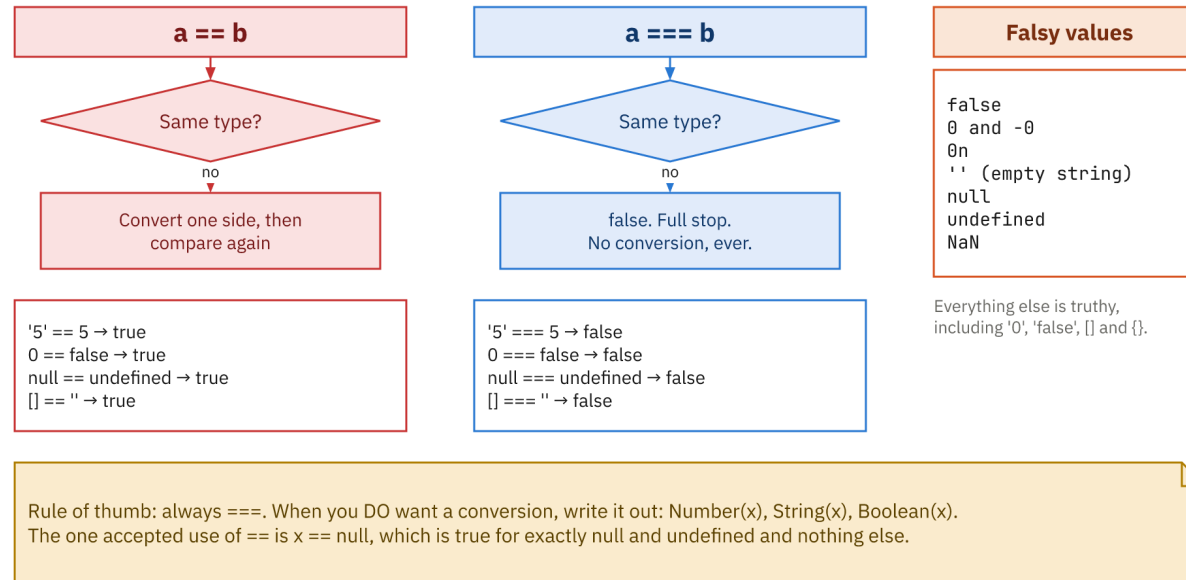
`${qty} bags = $$${qty * price}`; // "3 bags = $12"
`Tax: ${price * 0.07}.toFixed(2)} `; // "Tax: 0.28"
`You are ${age >= 18 ? "an adult" : "a minor"} `;
```

- Not just variables, any expression that produces a value
- Prefer template literals over `+` once there is more than one value

Coercion and Equality

Coercion and Equality

`==` converts before it compares. `===` never converts. Use `===` and convert on purpose.



- `==` converts before it compares; `===` never does
- Always `===`, and convert on purpose with `Number()` or `String()`